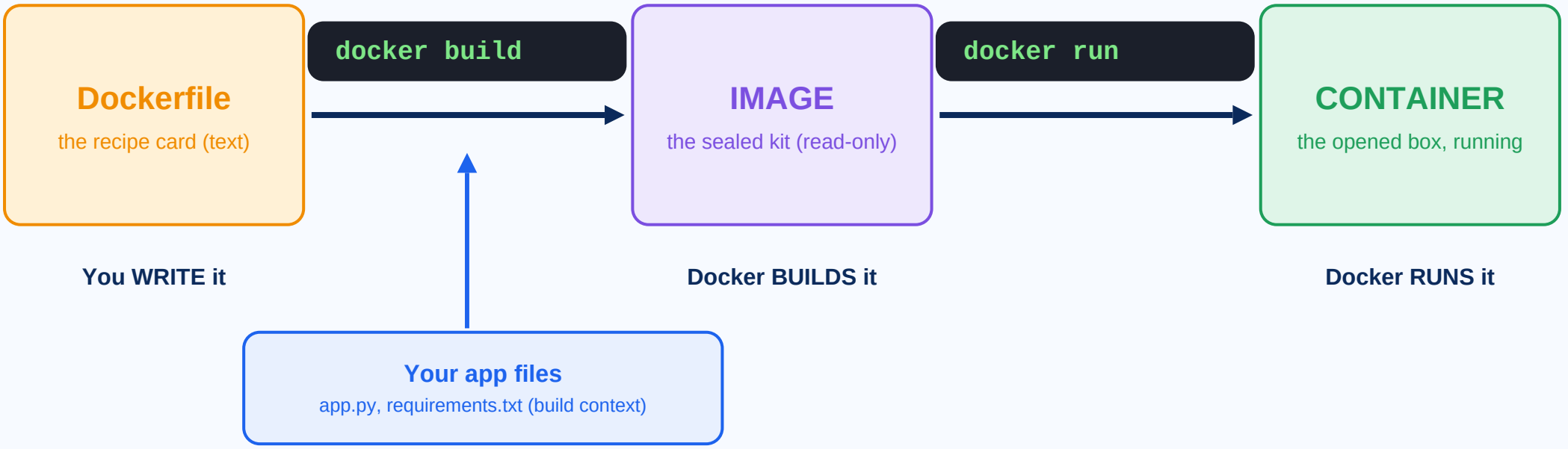
1. From Dockerfile to Running Container

Three artifacts, two commands

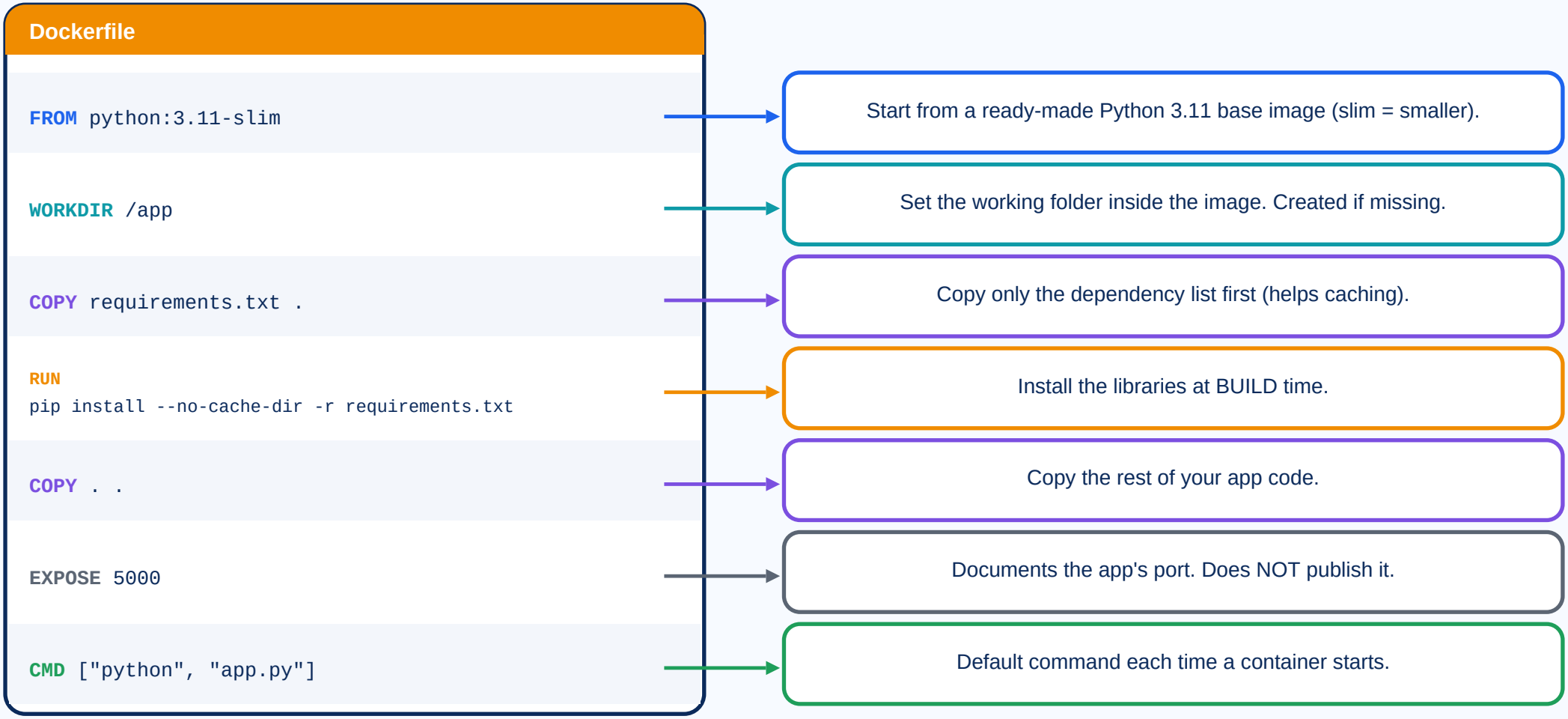


One Dockerfile can build many images. One image can start many containers.

Edit the Dockerfile -> rebuild the image -> start a new container.

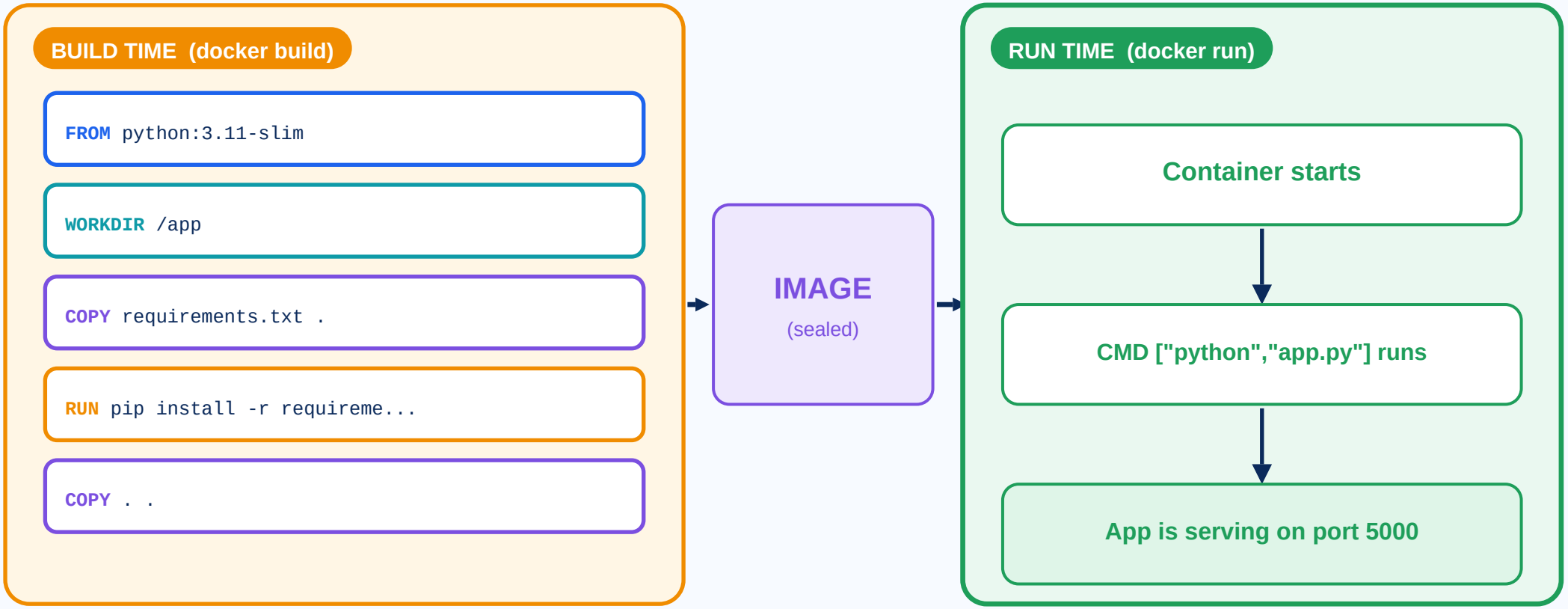
2. The Dockerfile, Line by Line

A small Flask app packed into an image



3. Build Time vs Run Time

RUN happens while building. CMD happens when a container starts.



ONCE per build

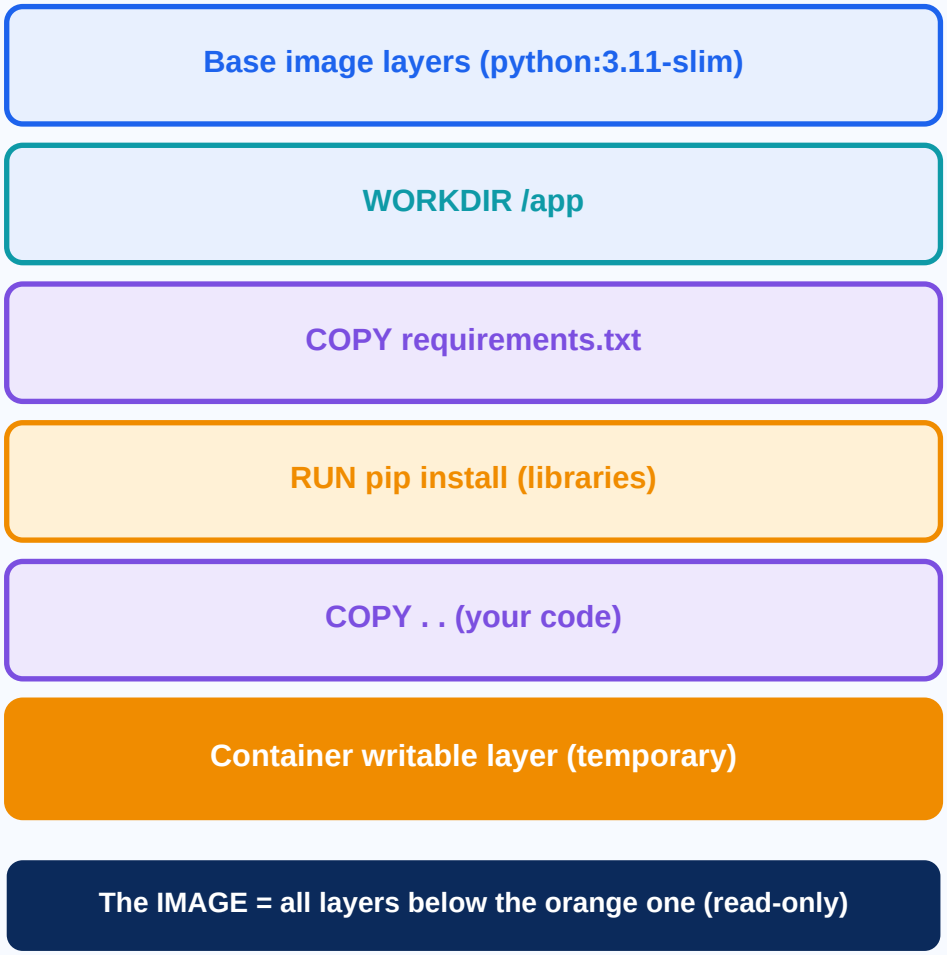
Result is saved inside the image

EVERY time a container starts

Nothing is saved back into the image

4. Image Layers

Each filesystem-changing instruction adds a read-only layer



Read-only and shared
Many containers can use the same image layers.

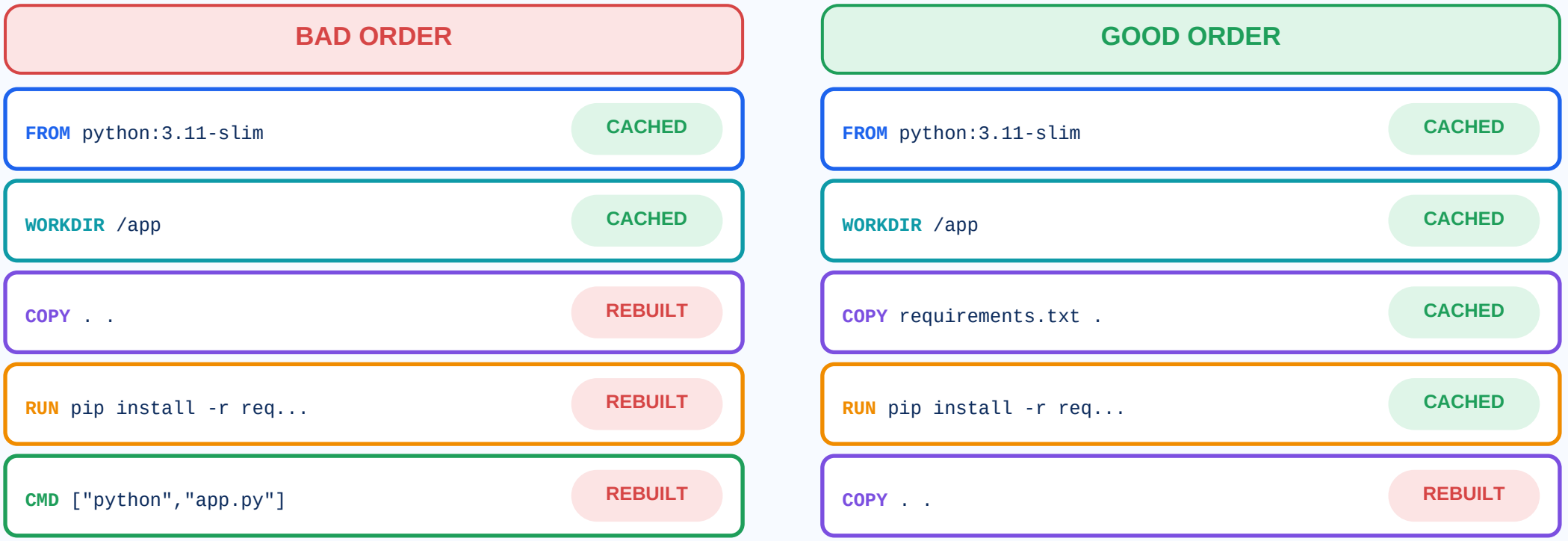
Container adds one thin writable layer
Files created while running go here.

Remove the container, lose that layer
Why data needs volumes (Episode 4).

See the layers yourself
docker history hello-flask:1.0

5. Layer Order Decides Build Speed

What happens on the 2nd build, after you change app.py



Rule: put what changes LEAST at the top, what changes MOST at the bottom.

Any code change breaks COPY . . so pip install runs AGAIN every build. Slow!

Only the code layer rebuilds. The pip layer is reused. Fast!

6. CMD vs ENTRYPOINT

What actually runs inside the container

	CMD only	ENTRYPOINT only	Both together
	CMD ["python","app.py"]	ENTRYPOINT ["python"]	ENTRYPOINT ["python"] CMD ["app.py"]
docker run IMG	**python app.py**	**python (interpreter starts)**	**python app.py**
docker run IMG <something else>	**REPLACES CMD:** python --version	**APPENDED:** python app.py	**REPLACES CMD only:** python other.py

Use --entrypoint on docker run to override an ENTRYPOINT.

Prefer the exec form ["a","b"] (JSON, double quotes).

7. Today's Lab: Build and Run Your First Image

Project folder and seven steps

```
hello-flask/
  app.py
  requirements.txt
  Dockerfile

app.py: Flask app, host=0.0.0.0
requirements.txt: flask==3.0.3
```

Tip for macOS

Port 5000 on the host is often used by AirPlay Receiver, so we map host 8080 to container 5000.

1. **Create**

```
mkdir hello-flask && cd hello-flask
```

2. **Write**

```
create app.py, requirements.txt, Dockerfile
```

3. **Build**

```
docker build -t hello-flask:1.0 .
```

4. **Check**

```
docker images
```

5. **Run**

```
docker run -d -p 8080:5000 --name hello hello-flask:1.0
```

6. **Browse**

```
open http://localhost:8080
```

7. **Clean**

```
docker stop hello && docker rm hello
```